TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS:

Byron Ruben Clemente Borbor

PARALELO:

A

SEMESTRE:

Quinto

ACTIVIDAD:

Actividad #1: Estudio de Caso - Sistema de Control de Rescate y Adopción Animal

PERIODO SEPTIEMBRE 2025 - ENERO 2026

Actividades a desarrollar en la Unidad 1

Actividad # 1

ENLACE A DIAGRAMA DE CLASES UML

https://lucid.app/lucidchart/8897699e-095d-4d2a-9504-8c6bb2edd7c4/edit?viewport_loc=-256%2C140%2C3072%2C1398%2C0_0&invitationId=inv_7dd3097d-c249-4787-a5a7-ac15e6435a7a

ENLACE A DIAGRAMA DE SECUENCIAS UML

https://lucid.app/lucidchart/310fbe1a-6a5e-4367-b62b-3cd7f5abe614/edit?viewport_loc=-921%2C-188%2C4608%2C2097%2C0_0&invitationId=inv_5d529346-0f19-4048-a0e6-72f65e8b738f

ENLACE A REPOSITORIO DE GITHUB

https://github.com/bclemente2210-cell/Actividad1SDI

1. Problema

En los refugios de animales se presentan diariamente casos de perros, gatos y otros animales que necesitan rescate, atención veterinaria o un hogar, pero cuando la información se registra manualmente o se encuentra en diferentes documentos muchas veces resulta difícil llevar un control de los animales ingresados, conocer su estado de salud, asignar responsables y dar seguimiento a los procesos de adopción.

Por ello se propone desarrollar un sistema de control de rescate y adopción animal, que permita registrar y administrar los casos de rescate desde el momento en que se recibe un reporte hasta que el animal es adoptado.

El sistema permitirá almacenar información de los animales, registrar su ubicación de rescate, asignar voluntarios y veterinarios, controlar tratamientos, gestionar adopciones y generar reportes estadísticos también facilitará la comunicación entre los ciudadanos que reportan casos y el personal encargado del refugio.

2. Objetivos

- Registrar los reportes de animales encontrados en situación de abandono, peligro, enfermedad o maltrato.
- Clasificar los animales según su especie, edad, sexo, estado de salud y otras características.
- Asignar voluntarios y veterinarios responsables de cada caso.
- Controlar las etapas del proceso de rescate y los tratamientos veterinarios.
- Gestionar las solicitudes de adopción y verificar el cumplimiento de los requisitos establecidos por el refugio.

- Generar reportes sobre rescates realizados, animales recuperados, adopciones y casos pendientes.

3. Actores principales

- **Ciudadano**

  La persona que informa sobre algún animal que requiera ayuda, puede registrar un reporte, indicar ubicación, describir la situación.

- **Voluntario**

  Se encarga de participar en los rescates, trasladar animales, proporcionar cuidados básicos autorizados y registrar las actividades realizadas y puede consultar los casos que le han sido asignados y actualizar su progreso.

- Veterinario

  Responsable de evaluar el estado de salud, registrar diagnósticos, prescribir tratamientos, programar controles y determinar si el animal está en condiciones de pasar a recuperación o adopción.

- **Administrador o secretario**

  Gestiona los usuarios, revisa los reportes, asigna personal, supervisa los casos, administra los registros de animales y controla las solicitudes de adopción, también puede consultar estadísticas y revisar el historial de modificaciones.

- **Adoptante**

  La persona interesada en brindar un hogar a un animal, puede presentar una solicitud, registrar sus datos, aportar la información requerida y consultar el estado de su proceso de adopción.

4. Requerimientos funcionales

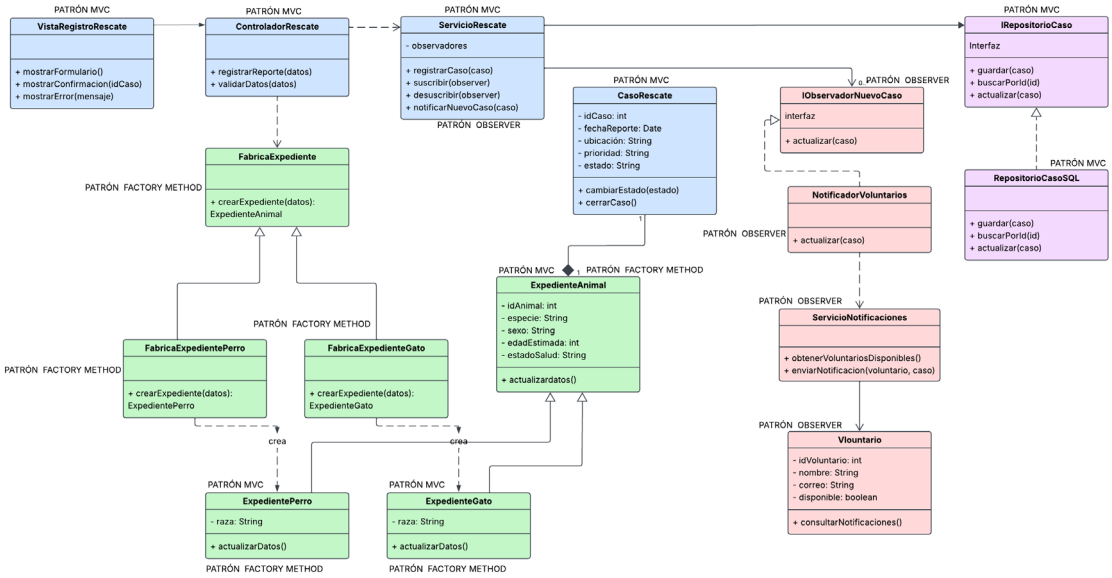
Código	Requerimiento funcional	Descripción
RF01	Registrar reportes	Permitir que los ciudadanos informen sobre perros o gatos que necesitan ayuda.
RF02	Crear casos de rescate	Generar un código único para cada caso registrado.
RF03	Registrar animales	Guardar información como especie, raza, sexo, edad aproximada, color y características físicas.
RF04	Clasificar especies	Categorizar los animales en perros y gatos.
RF05	Registrar ubicación	Guardar la dirección, el sector y la fecha en que se encontró al animal.
RF06	Establecer prioridad	Clasificar los casos según su urgencia, riesgo y estado de salud.

RF07	Asignar responsables	Permitir que el administrador asigne voluntarios y veterinarios a cada caso.
RF08	Gestionar estados	Actualizar el estado del animal durante las diferentes etapas del proceso.
RF09	Registrar atención veterinaria	Almacenar diagnósticos, tratamientos, vacunas, medicamentos y controles médicos.
RF10	Controlar recursos	Registrar espacios disponibles, alimentos, medicamentos y materiales utilizados.
RF11	Gestionar adopciones	Registrar solicitudes, evaluar candidatos, aprobar o rechazar solicitudes y documentar la entrega del animal.
RF12	Programar actividades	Agendar consultas veterinarias, vacunaciones, rescates y seguimientos posteriores a la adopción.
RF13	Registrar evidencias	Permitir adjuntar fotografías, documentos y observaciones relacionadas con cada caso.
RF14	Enviar notificaciones	Informar sobre asignaciones, cambios de estado, citas y solicitudes pendientes.
RF15	Consultar y buscar	Buscar animales por código, especie, estado, ubicación o fecha de ingreso.
RF16	Generar reportes	Mostrar estadísticas sobre rescates, tratamientos, adopciones y casos pendientes.
RF17	Administrar usuarios	Crear cuentas y establecer permisos según el rol de cada usuario.
RF18	Registrar traslados	Documentar los traslados de animales entre refugios o centros veterinarios.
RF19	Dar seguimiento	Registrar visitas, observaciones y novedades después de una adopción.
RF20	Consultar historial	Mostrar las modificaciones, los responsables y las actividades realizadas en cada caso.

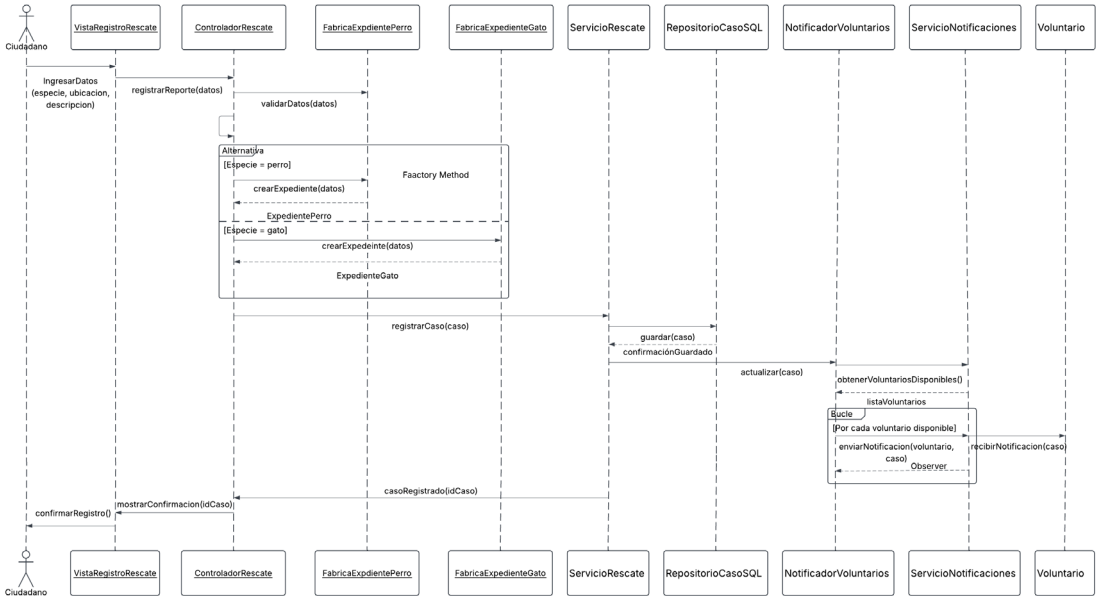
5. Requerimientos no funcionales

Código	Requerimiento no funcional	Descripción
RNF01	Escalabilidad	Permitir aumentar la cantidad de usuarios, animales y reportes sin necesidad de rediseñar todo el sistema.
RNF02	Mantenibilidad	Organizar el software en módulos para facilitar las correcciones y futuras mejoras.
RNF03	Trazabilidad de versiones	Mantener un control de las versiones del código fuente y registrar los cambios realizados durante el desarrollo.
RNF04	Usabilidad	Presentar una interfaz sencilla, clara y fácil de utilizar para los ciudadanos y el personal del refugio.
RNF05	Recuperación ante fallos	Facilitar la restauración de los registros a partir de copias de seguridad cuando ocurra un incidente.

6. Diagramas de clases UML



7. Diagrama Secuencial UML



8. Aplicación de patrones de diseño

- **MVC (Modelo-Vista-Controlador)**

Separar la interfaz gráfica, el procesamiento de las solicitudes y los datos del sistema, en el sistema de rescate, VistaRegistroRescate muestra el formulario que utiliza el ciudadano. ControladorRescate recibe los datos y valida la solicitud y, ServicioRescate coordina el registro del caso y las clases de modelo representan la información del animal y del rescate.

Este patrón facilita el mantenimiento, ya que los cambios en el formulario no tienen que modificar directamente las clases encargadas de almacenar los casos.

- **Factory Method**

Crea objetos mediante métodos de fábrica, evitando que el controlador tenga que construir directamente cada tipo de expediente.

La clase abstracta FabricaExpediente define el método crearExpediente(). Sus implementaciones concretas, FabricaExpedientePerro y FabricaExpedienteGato, crean los respectivos expedientes.

Esto permite que cada tipo de animal tenga sus datos y comportamientos específicos. Además, facilita agregar nuevos tipos de expedientes en el futuro sin concentrar toda la lógica de creación en una sola clase.

- **Observer**

Notifica automáticamente a los componentes interesados cuando se registra un nuevo caso de rescate, en este sistema ServicioRescate actúa como publicador del evento, mantiene una lista de observadores suscritos y cuando guarda un nuevo caso, notifica a NotificadorVoluntarios mediante la interfaz IObservadorNuevoCaso.

El notificador consulta cuáles voluntarios están disponibles y solicita el envío de las notificaciones correspondientes, asi el servicio encargado de registrar los casos no necesita conocer los detalles del envío de mensajes.

9. Principios SOLID

SRP: El controlador valida solicitudes, la fábrica crea expedientes y el notificador envía avisos.

OCP: Se pueden agregar nuevas fábricas concretas o tipos de observadores sin modificar las abstracciones existentes.

LSP: Las fábricas concretas deben cumplir el contrato establecido por FabricaExpediente, y los expedientes deben respetar el comportamiento de ExpedienteAnimal.

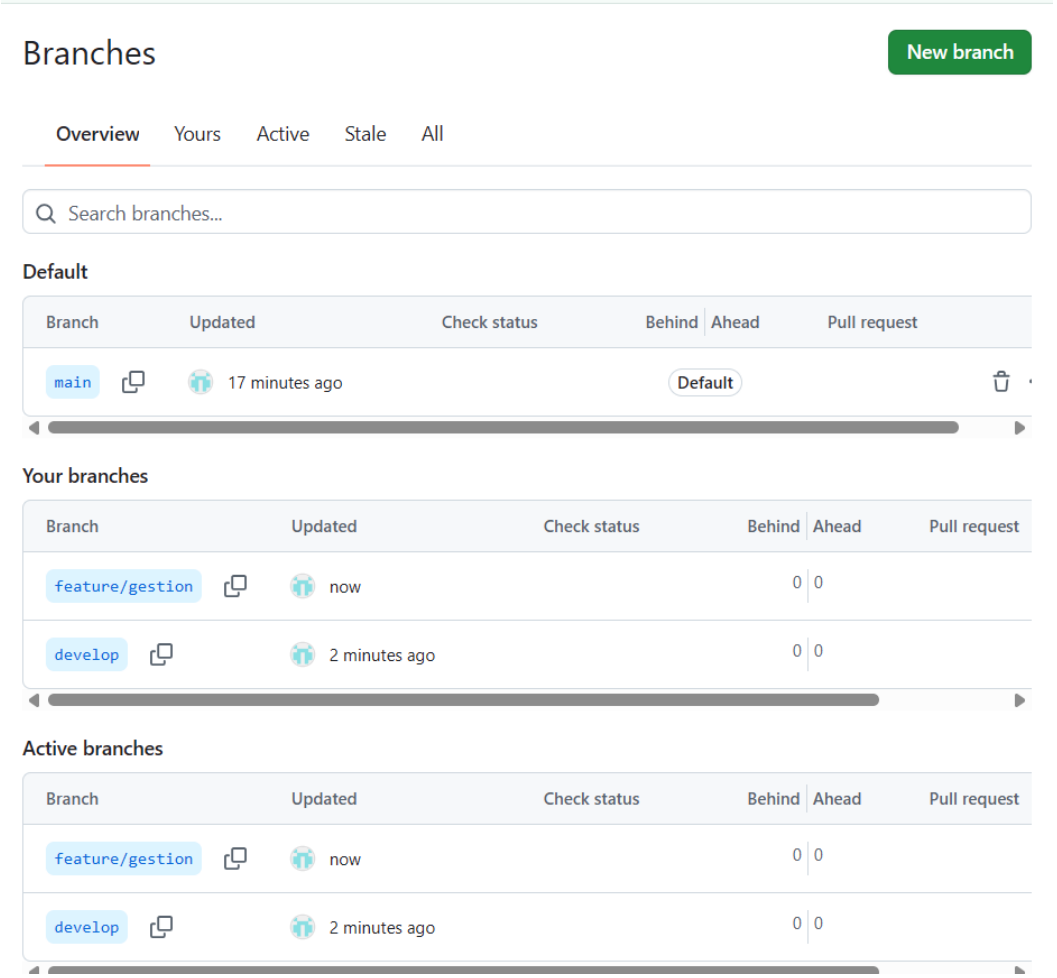
ISP: La interfaz IObservadorNuevoCaso contiene únicamente la operación necesaria para recibir un aviso de nuevo caso.

DIP: El controlador utiliza la abstracción FabricaExpediente y el servicio depende de IRepositorioCaso, no de una implementación concreta.

10. Justificacion de Gitflow

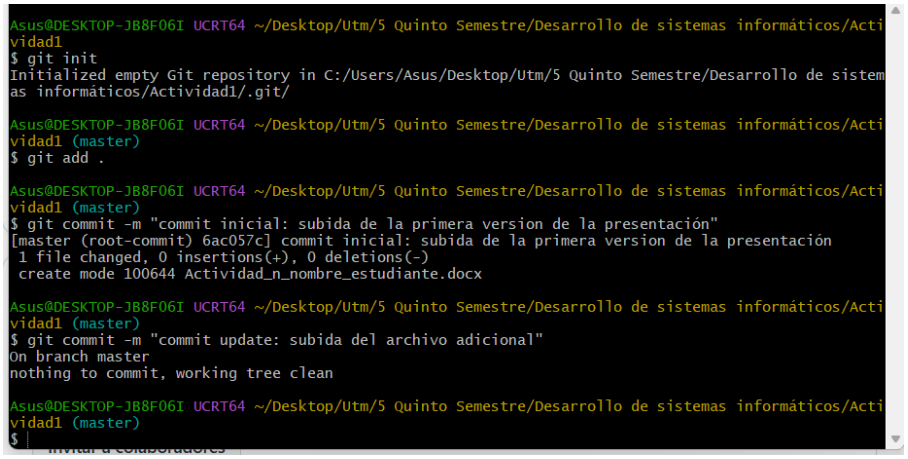
Es una estrategia de organización y control de versiones, ya que permite mantener el código ordenado y facilitar el trabajo durante las diferentes etapas del proyecto.

La rama master almacenará las versiones estables del sistema, mientras que la rama develop integrará los avances antes de incorporarlos a la versión final, se utilizarán ramas feature/ para desarrollar funcionalidades específicas, como el registro de casos de rescate, la gestión de expedientes de perros y gatos y las solicitudes de adopción, sin modificar directamente el código estable, esta organización permite identificar los cambios realizados, corregir errores con mayor facilidad, revisar las modificaciones antes de integrarlas y mantener un historial de las versiones del proyecto en GitHub. De esta manera, Gitflow contribuye a la mantenibilidad, trazabilidad de versiones y recuperación ante fallo en un futuro, en esta primera actividad todos los detalles del análisis del caso se encuentran en el documento pdf.

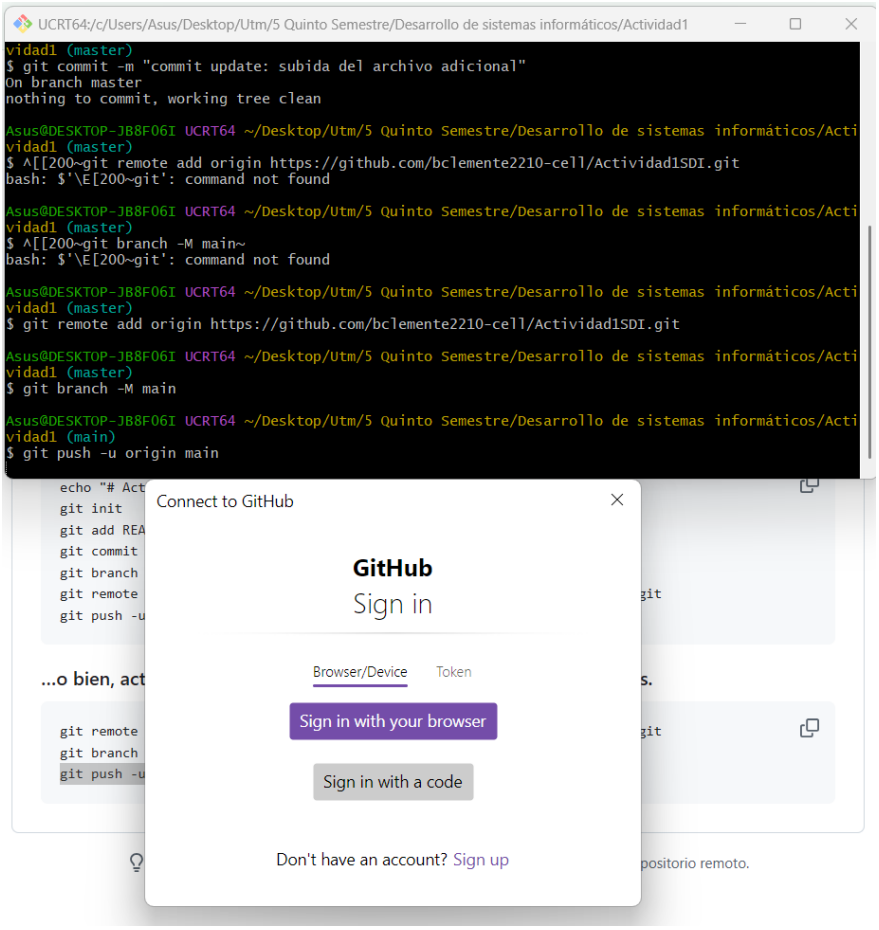


11. Registro de avance usando COMMITS

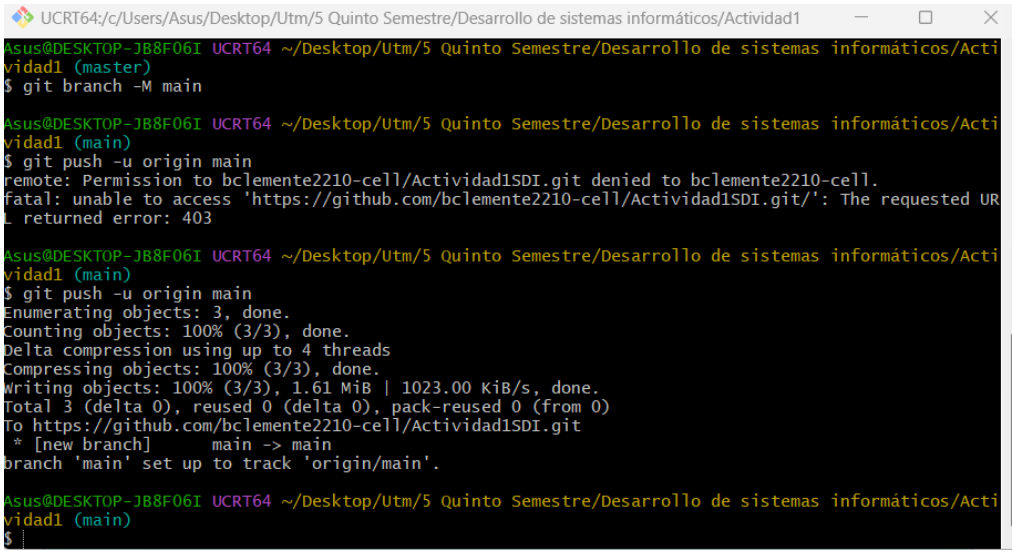
Realizamos la preparación y la creación del proyecto, y subimos nuestro primer archivo usando git add . y git commit



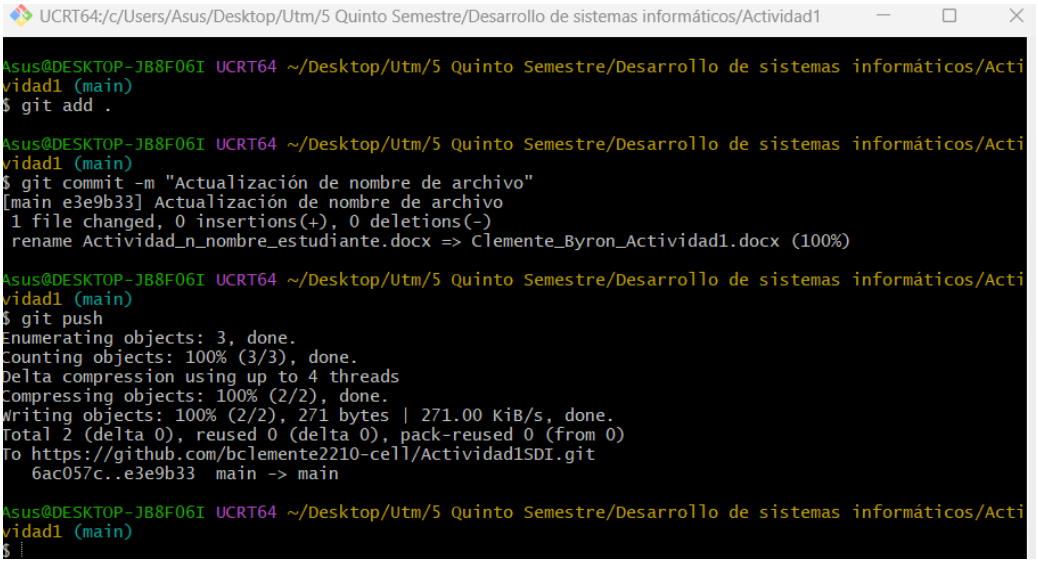
Nos registramos correctamente con nuestra cuenta de GitHub usando token



Subimos nuestro archivo a GitHub(La nube)



Usamos git commit -m para “actualizar nombre de archivo” y asi registrar una actualización



12. Conclusiones

El Sistema de Control de Rescate y Adopción Animal utilizará tres patrones de diseño, MVC, Factory Method y Observer. MVC permite organizar la presentación, el procesamiento y los datos, mientras que Factory Method facilita crear expedientes específicos para perros y gatos, y Observer permite comunicar automáticamente los nuevos casos a los voluntarios.

Estos patrones contribuyen a separar responsabilidades y reducir el acoplamiento entre las clases. Junto con los principios SOLID y el flujo de trabajo Gitflow, proporcionan una base organizada para desarrollar, mantener y ampliar el sistema.

ENLACE A DIAGRAMA DE CLASES UML

https://lucid.app/lucidchart/8897699e-095d-4d2a-9504-8c6bb2edd7c4/edit?viewport_loc=-256%2C140%2C3072%2C1398%2C0_0&invitationId=inv_7dd3097d-c249-4787-a5a7-ac15e6435a7a

ENLACE A DIAGRAMA DE SECUENCIAS UML

https://lucid.app/lucidchart/310fbe1a-6a5e-4367-b62b-3cd7f5abe614/edit?viewport_loc=-921%2C-188%2C4608%2C2097%2C0_0&invitationId=inv_5d529346-0f19-4048-a0e6-72f65e8b738f

ENLACE A REPOSITORIO DE GITHUB

https://github.com/bclemente2210-cell/Actividad1SDI